Spis treści

1. **Wymagania użytkownika** 2
2. **Diagram przypadków użycia** 4
3. **Analityczny diagram klas przed analizą dynamiczną** 5
4. **Scenariusz przypadku użycia** 6
5. **Diagram Aktywności** 7
6. **Diagram Stanów** 8
7. **Projekt GUI** 9
8. **Analiza Dynamiczna** 18
9. **Analityczny diagram klas po analizie dynamicznej** 19
10. **Decyzje projektowe** 20
11. **Projektowy diagram klas** 22

1. Wymagania użytkownika

Gildia odkrywców potrzebuje bazy danych do przechowywania informacji na temat odkrywców zapisanych w gildii oraz do planowania eksploracji.

1.1. Gildia chce przechowywać informacje o imieniu, rasie i miejscu urodzenia odkrywcy. Chcemy także pamiętać jego statystyki siły, zręczności, inteligencji oraz posiadane doświadczenie.

1.2. Odkrywcy specjalizują się w jednej z czterech klas: mag, dla którego pamiętamy dodatkowo jego zasoby many, znane czary (mag musi znać min. 1 czar) oraz nazwę akademii, z której pobierał nauki; wojownik, pamiętamy dodatkowo jego statystykę obronny oraz preferowany typ broni; zwiadowca, pamiętamy dodatkowo jego statystykę celności i zdolności zwiadowczych; templariusz będący połączeniem maga oraz wojownika, dla którego pamiętamy statystykę zespojenia z maną oraz moc emitowanej aury.

1.3. Każdy odkrywca posiada również statystykę poziomu mocy, która obliczana jest na podstawie siły, zręczności, doświadczenia oraz dodatkowych parametrów w zależności od klasy.

1.4. Gildia przechowuje informacje o każdej znanej broni (nazwa, opis, typ broni, rzadkość oraz wymagane statystyki siły, zręczności i inteligencji potrzebne do dzierżenia tej broni). Każda broń może mieć tylko jednego właściciela, a odkrywca może posiadać kilka broni.

1.5. Każdy odkrywca może pracować w kilku profesjach (unikalna nazwa, opis). Określona jest jedna płaca minimalna dla wszystkich profesji. Dla każdej pracy chcemy pamiętać datę rozpoczęcia tej pracy, datę zakończenia pracy (jeżeli odkrywca już nie pracuje w danej profesji), ekspertyzę w tej profesji, a także jego płacę. Gildia archiwizuje informacje o każdej pracy wykonywanej przez odkrywcę.

1.6. Przechowywane są również informacje o znanych krainach (unikalna nazwa, dominujący żywioł, wielkość, populacja). Każda kraina zawiera w sobiem przynajmniej jedną starożytną ruinę (unikalna nazwa, wiek, poziom trudności oraz poziom mocy wymagany do eksploracji ruin). Każda ruina posiada jednego lub więcej starożytnych, czyli istotę chroniącą ruiny (imię, opis, życia, siła ataku, lista umiejętności (musi posiadać co najmniej jedną)). Każdy starożytny może dodatkowo być powietrznym starożytnym (rozmiar skrzydeł, prędkość lotu) i/lub przeklętym starożytnym (moc klątwy, rozmiar przeklętej aury).

1.7. Zwiadowca może posiadać wiedzę na temat wielu krain. Chcemy wtedy pamiętać poziom jego wiedzy na temat każdej znanej przez niego krainy.

1.8. Do każdej starożytnej ruiny może być przeprowadzona ekspedycja. Dla każdej ekspedycji jest pamiętana jej nazwa, status (zaplanowany, rozpoczęty, zakończony, anulowany), oraz data rozpoczęcia i zakończenia.

1.9. System daje dostęp do następujących funkcjonalności:

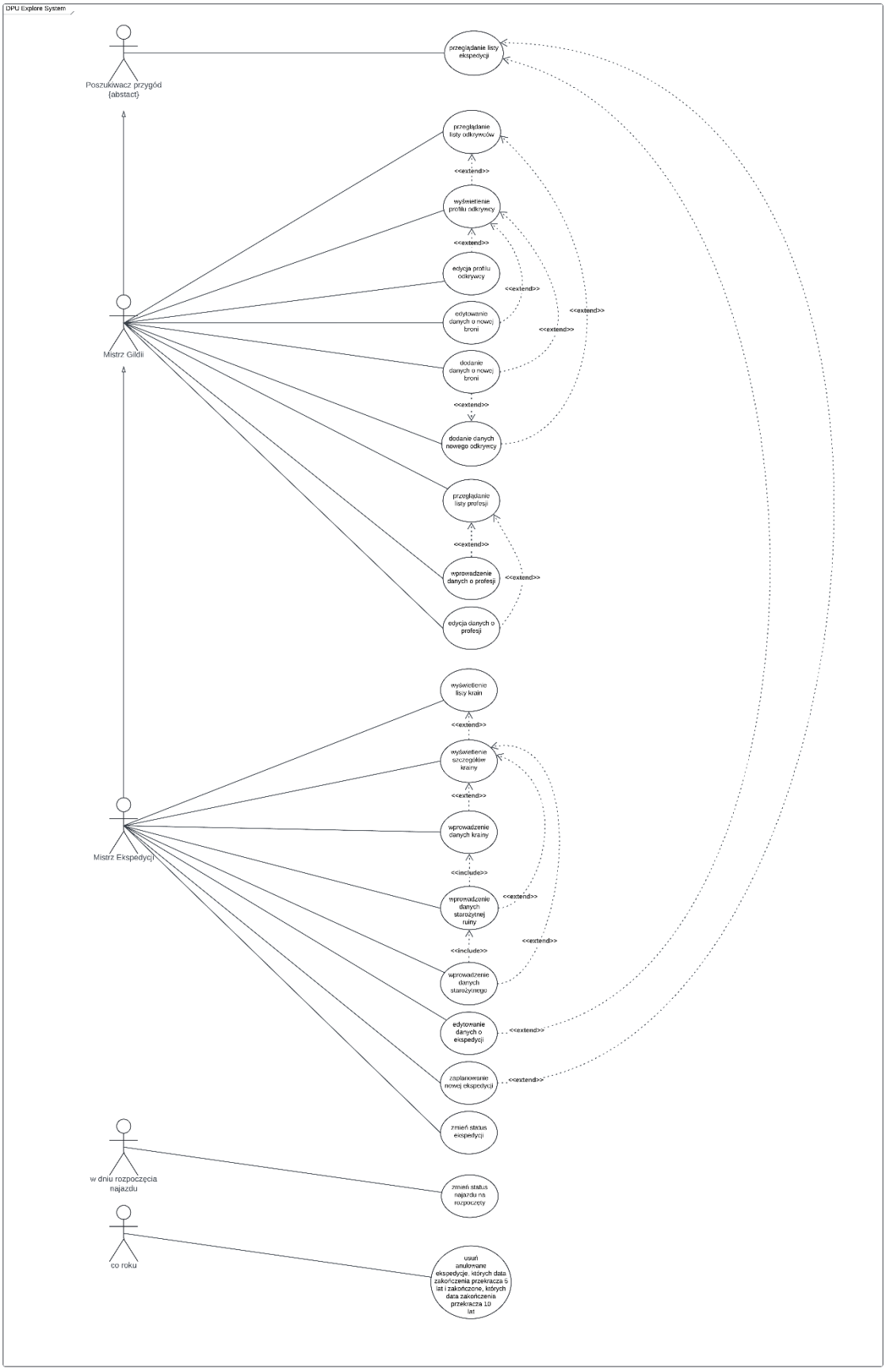
1.9.1. Każdy odkrywca może przeglądać listę ekspedycji,

1.9.2. Mistrz gildii może dodawać, edytować i dane o odkrywcach oraz profesjach i broni,

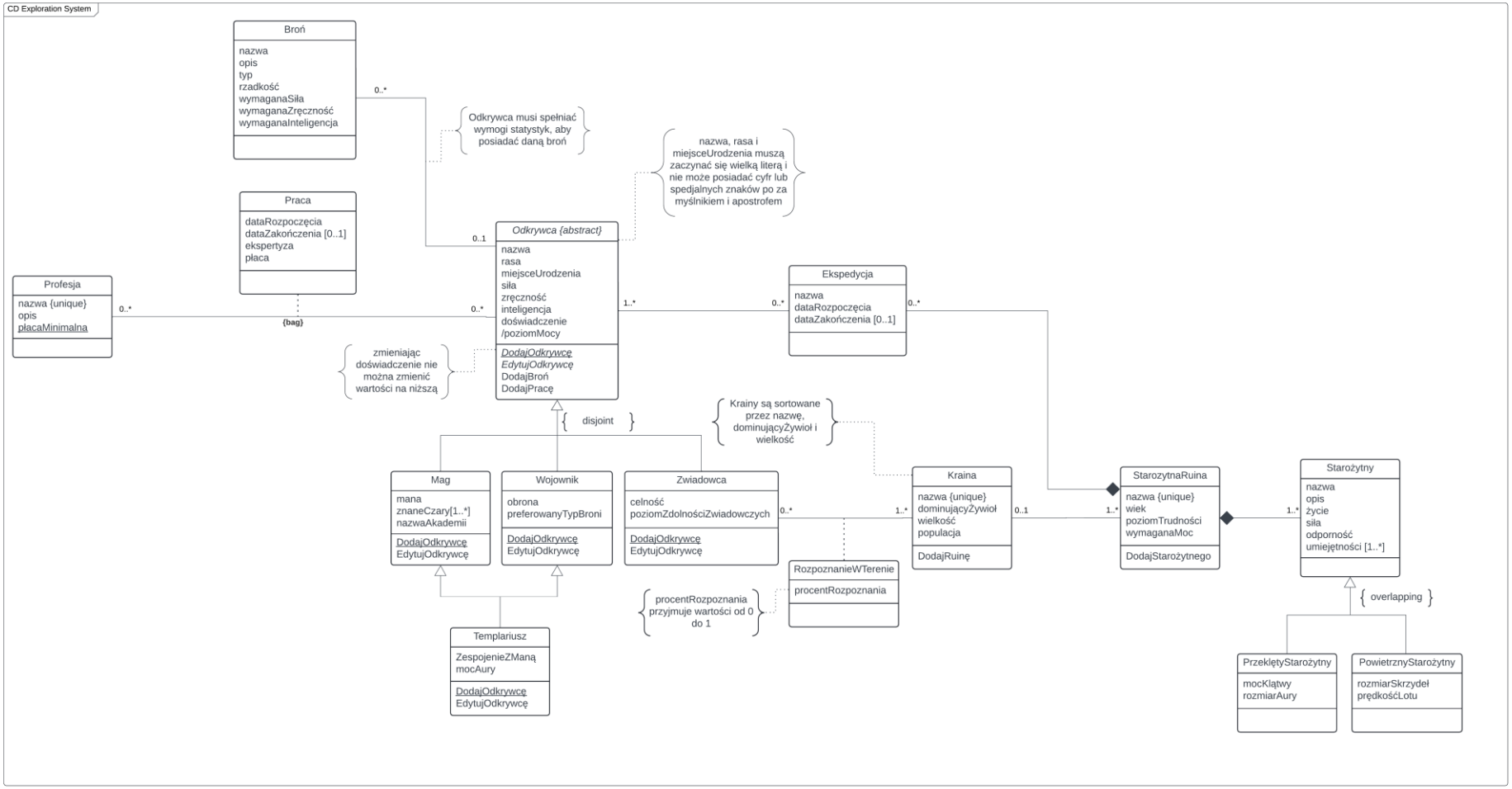
1.9.3. Mistrz ekspedycji może dodawać, edytować dane o krainach i ich starożytnych ruinach oraz o starożytny znajdujących się w tych ruinach. Może również planować nowe ekspedycje i edytować dane o nich oraz może zmienić ich status. Z poziomu planowania ekspedycji jest możliwe dodanie nowego odkrywcy,

1.9.4. W dniu rozpoczęcia ekspedycji system automatycznie zmienia jej status,
1.9.5. Co roku system usuwa ekspedycja, których data zakończenia przekracza 10 lat, kiedy są zakończone lub 5 lat kiedy są anulowane.

2. Diagram przypadków użycia



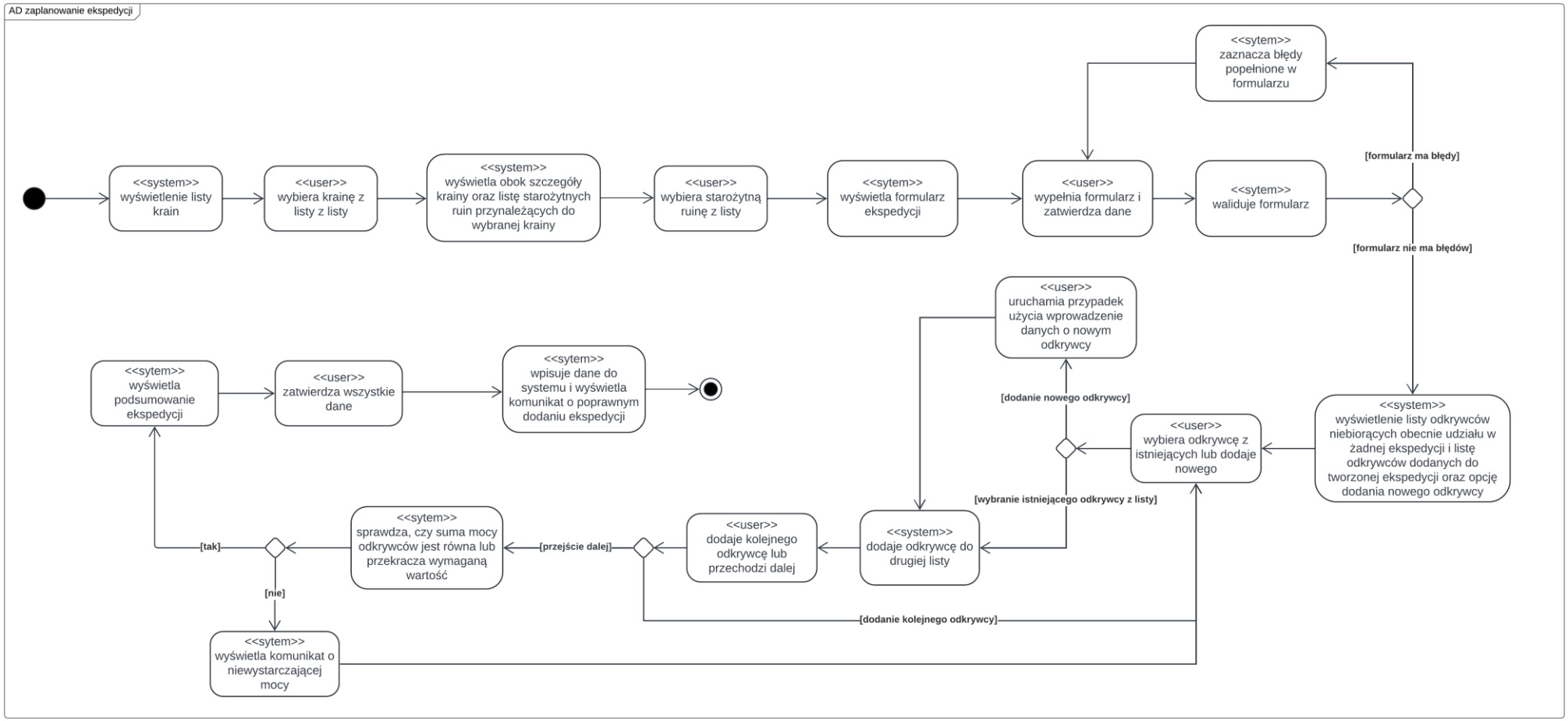
3. Analityczny diagram klas przed analizą dynamiczną



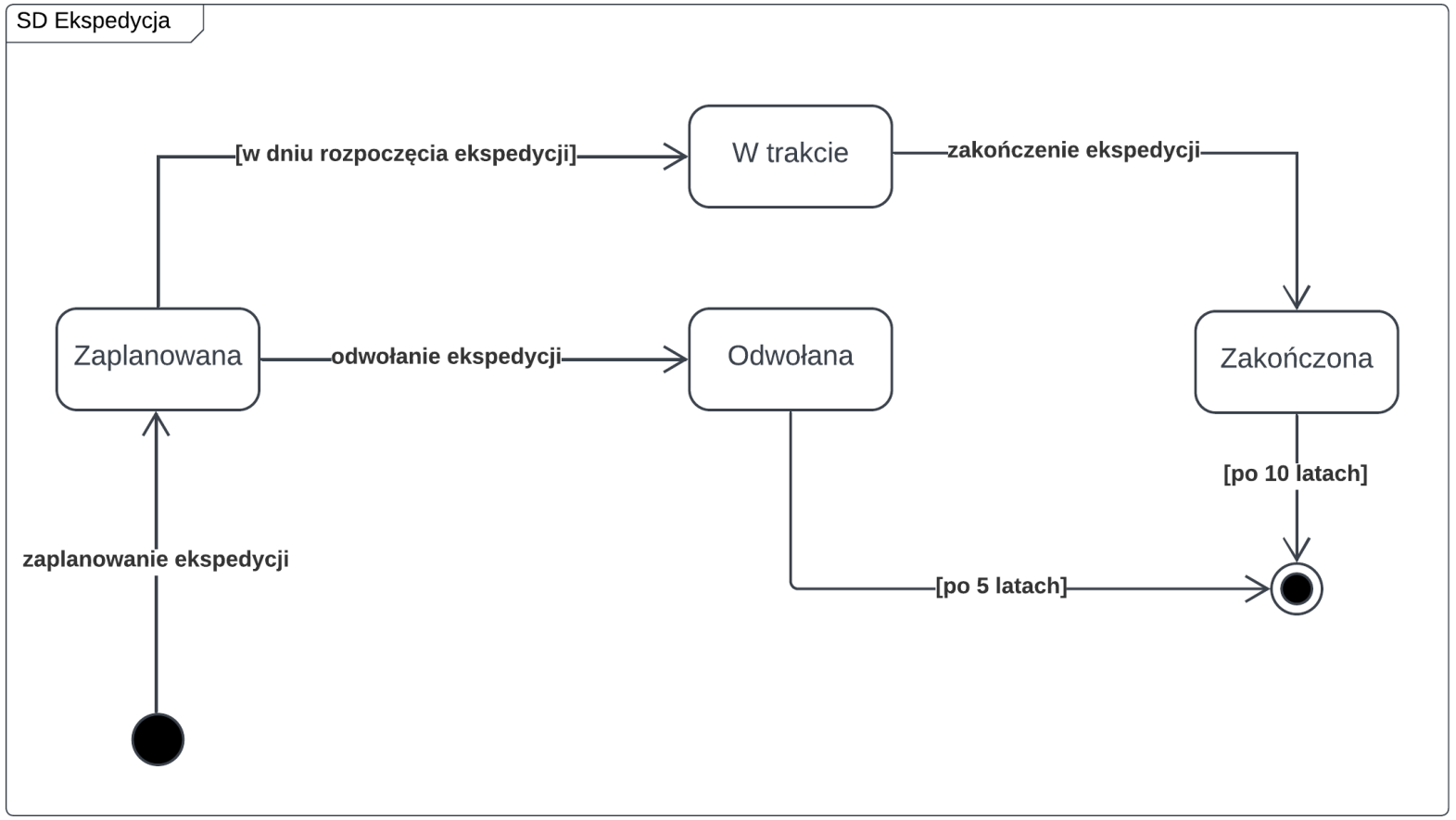
4. Scenariusz przypadku użycia

Nazwa przypadku użycia	Zaplanowanie nowej ekspedycji
Warunek początkowy	W bazie jest dostępna 1 kraina
Główny przepływ zdarzeń	1. Aktor uruchamia przypadek użycia. 2. System wyświetla listę krain. 3. Aktor wybiera krainę z listy. 4. System wyświetla szczegóły krainy i drugą listę starożytnych ruin przynależących do wybranej krainy. 5. Aktor wybiera starożytną ruinę z listy. 6. System wyświetla formularz ekspedycji. 7. Aktor wypełnia formularz i zatwierdza dane. 8. Jeżeli wszystkie dane są poprawne system wyświetla listę odkrywców, którzy nie biorą udziału w żadnej ekspedycji i drugą, pustą listę odkrywców przypisanych do tworzonej ekspedycji oraz opcję dodania nowego odkrywcy do ekspedycji. 9. Aktor wybiera odkrywcę z pierwszej listy. 10. System dodaje odkrywcę do drugiej listy. 11. Aktor wybiera opcję przejścia dalej. 12. System wyświetla podsumowanie ekspedycji 13. Aktor zatwierdza wszystkie dane 14. System dodaje ekspedycję do bazy i wyświetla stosowny komunikat
Alternatywne przepływy zdarzeń	7a. Formularz zawiera niepoprawne dane. System wyświetla komunikat błędu i pokazuje, w których miejscach jest błąd. 9a. Aktor dodaje nowego odkrywcę i uruchamia przypadek użycia „dodanie nowego odkrywcy”. Po zakończeniu tego przypadku użycia przechodzi do punktu 10 11a. Aktor chce dodać kolejnego odkrywcę. Wraca do punktu 9.
Zakończenie	Zakończenie w dowolnym momencie
Warunek końcowy	W przypadku poprawnego utworzenia ekspedycji do systemu zostają wprowadzone informacje: nazwa, data rozpoczęcia, dane eksplorowanych ruin, dane odkrywców biorących udział w ekspedycji i status – zaplanowana)

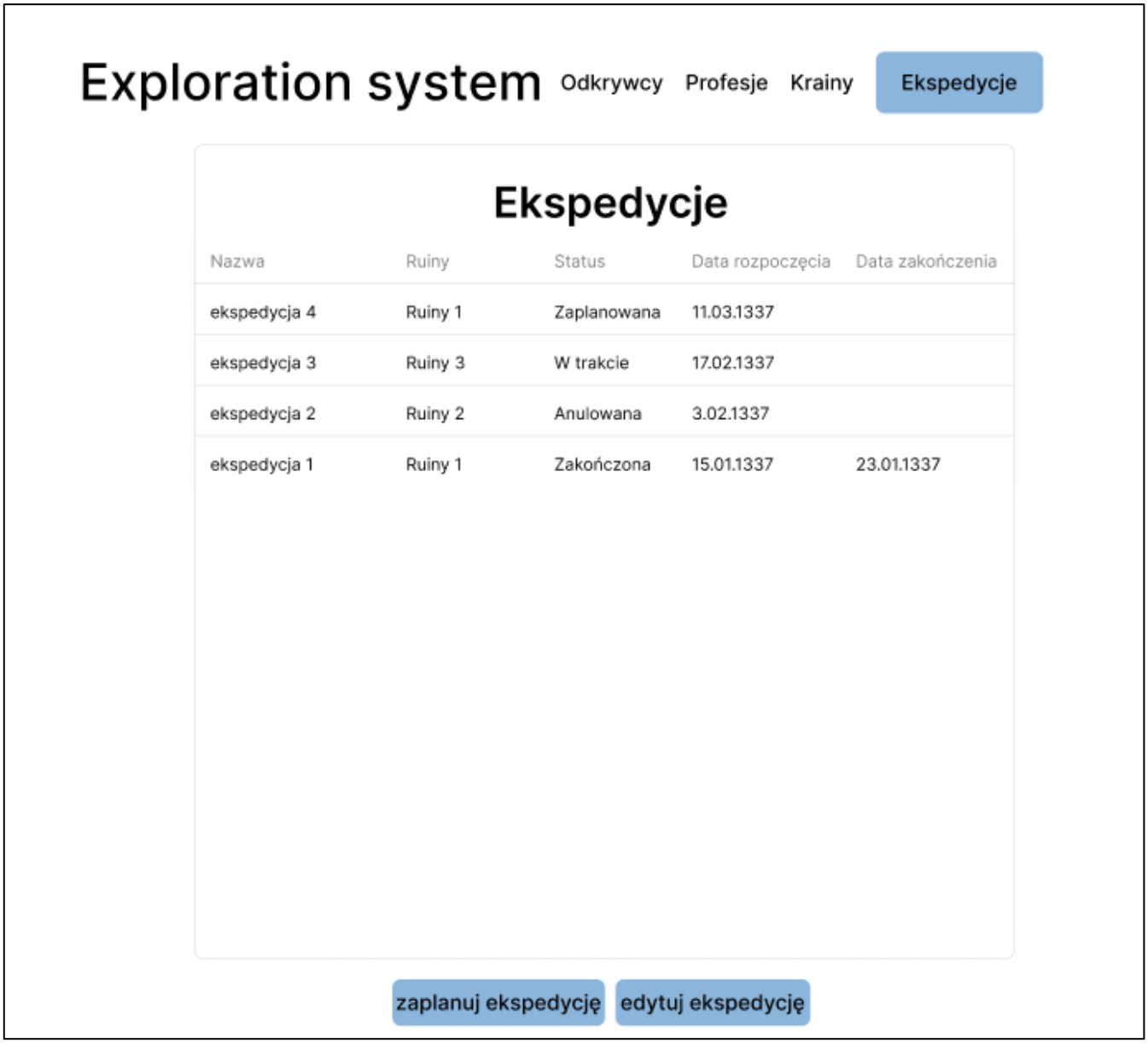
5. Diagram Aktywności



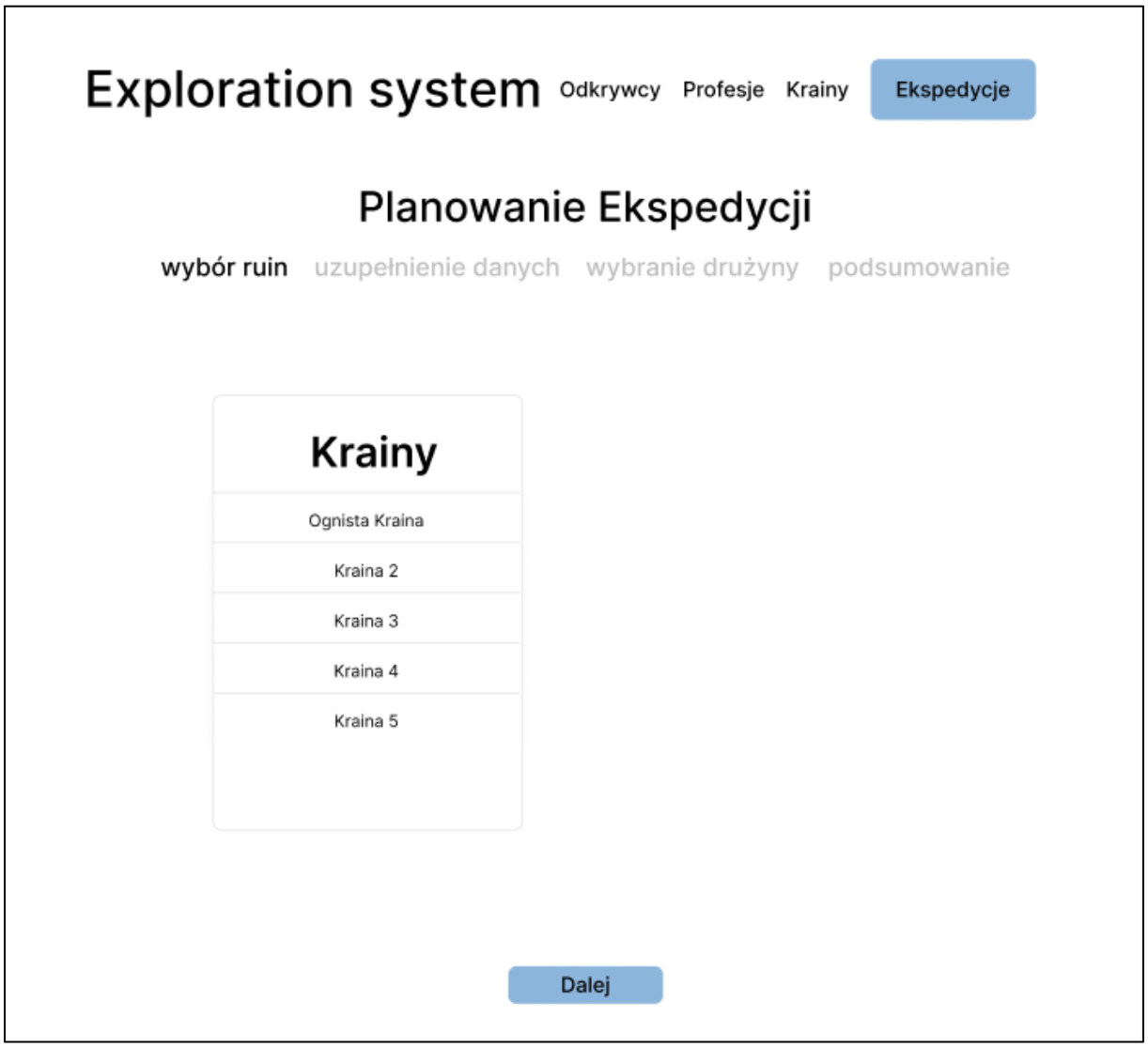
6. Diagram Stanów



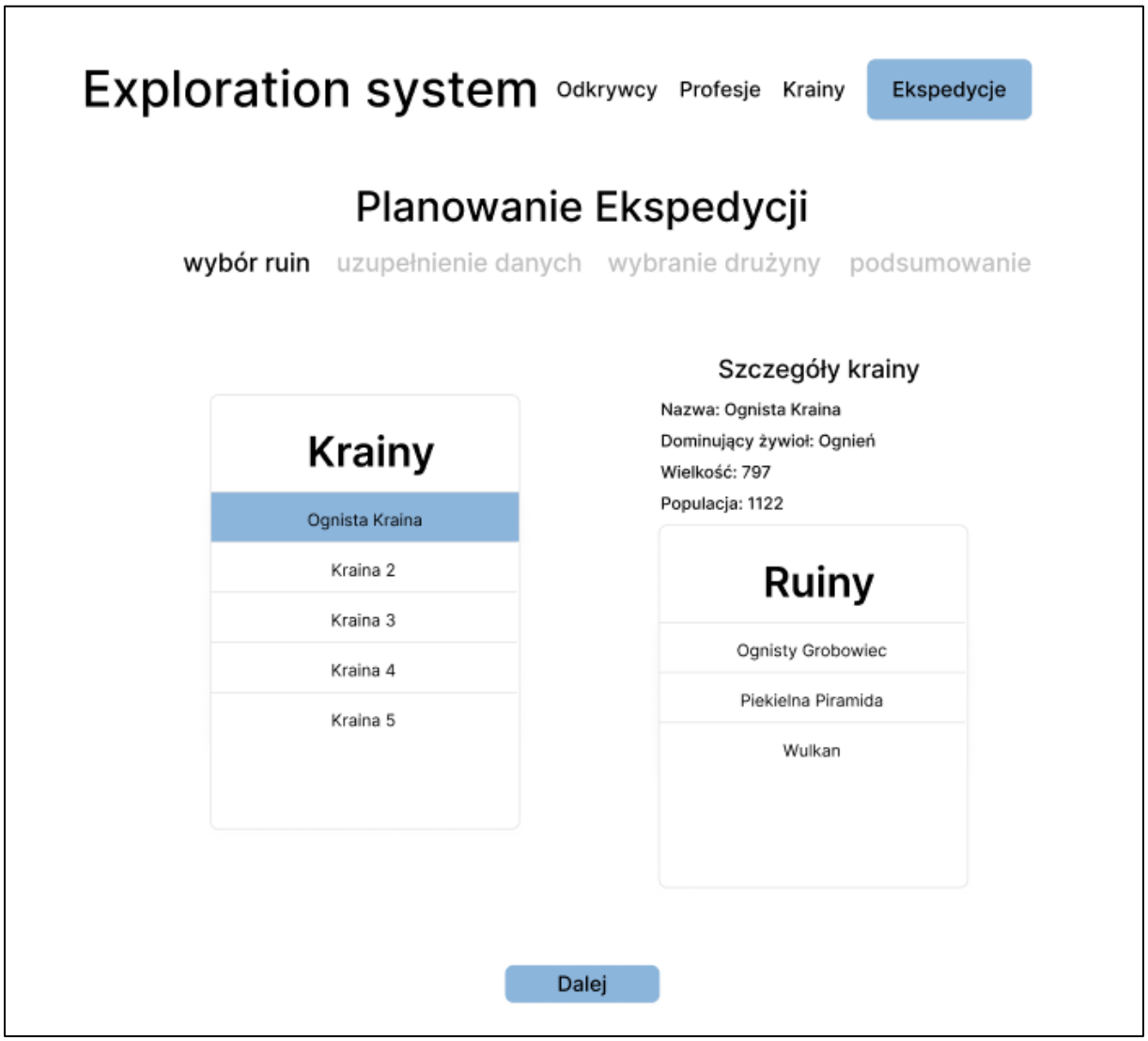
7. Projekt GUI



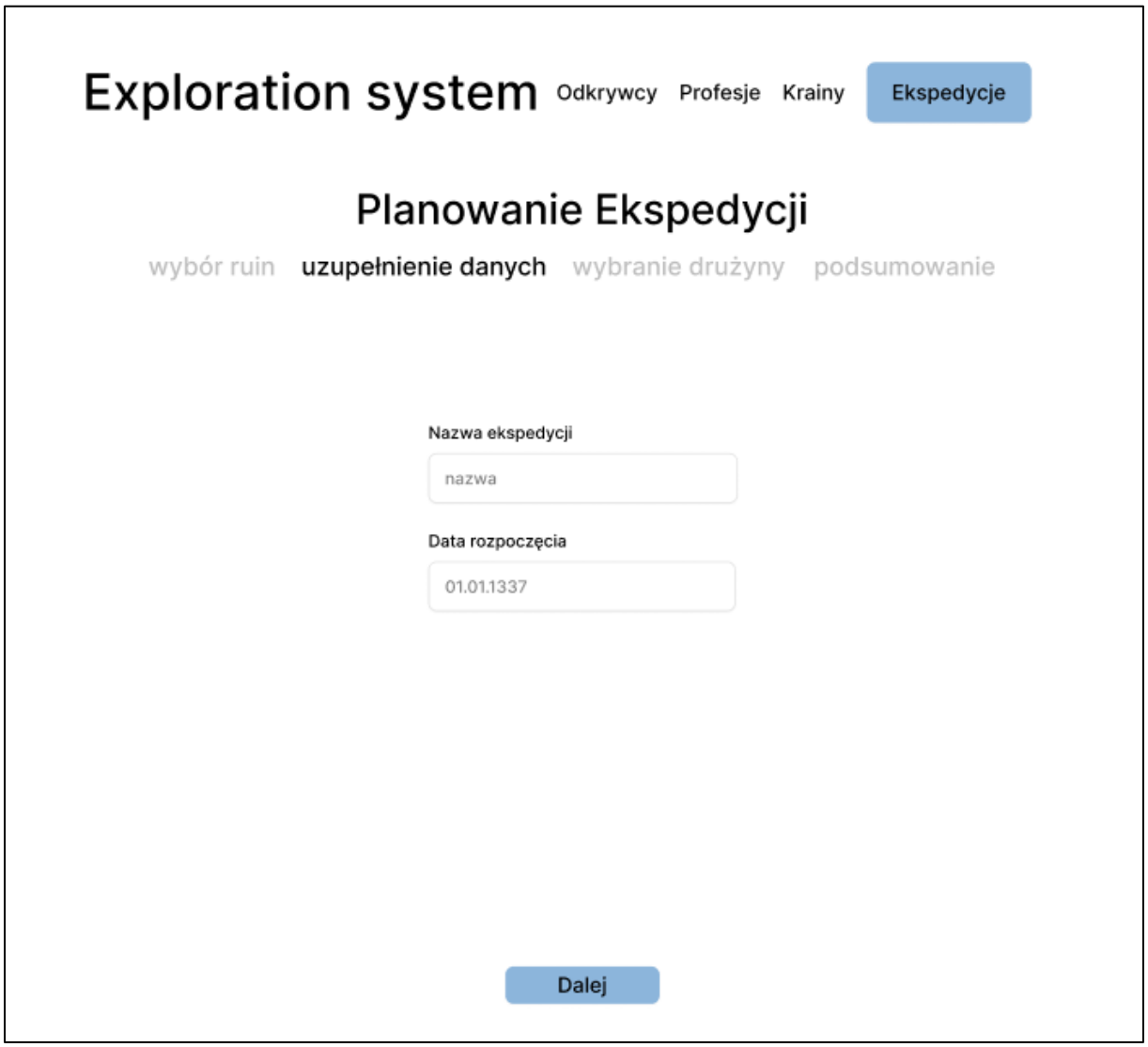
Rysunek 1: Widok listy eksploracji



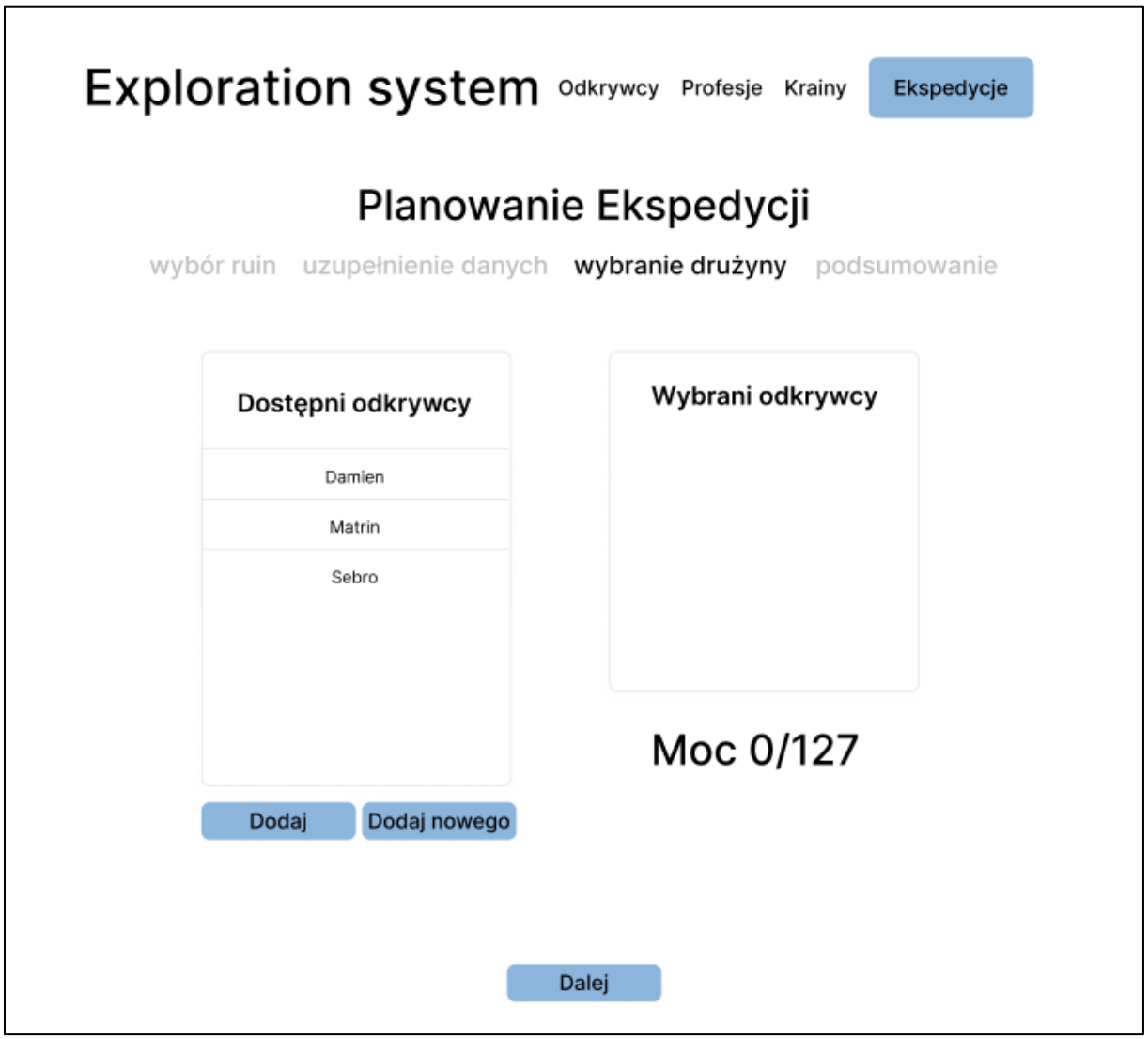
Rysunek 2: Widok planowania ekspedycji - wybór krainy



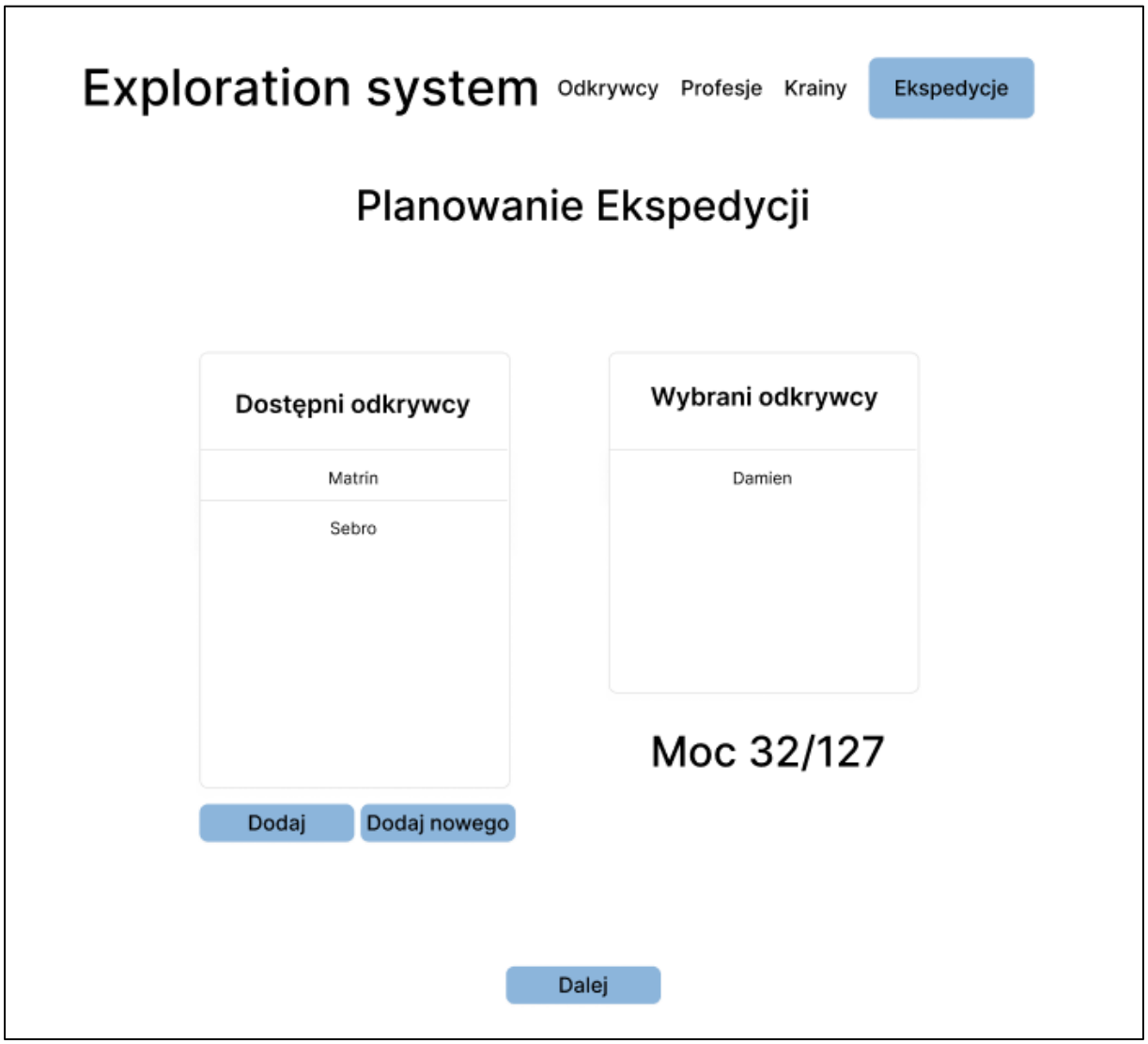
Rysunek 3: Widok planowania ekspedycji - wybór ruin



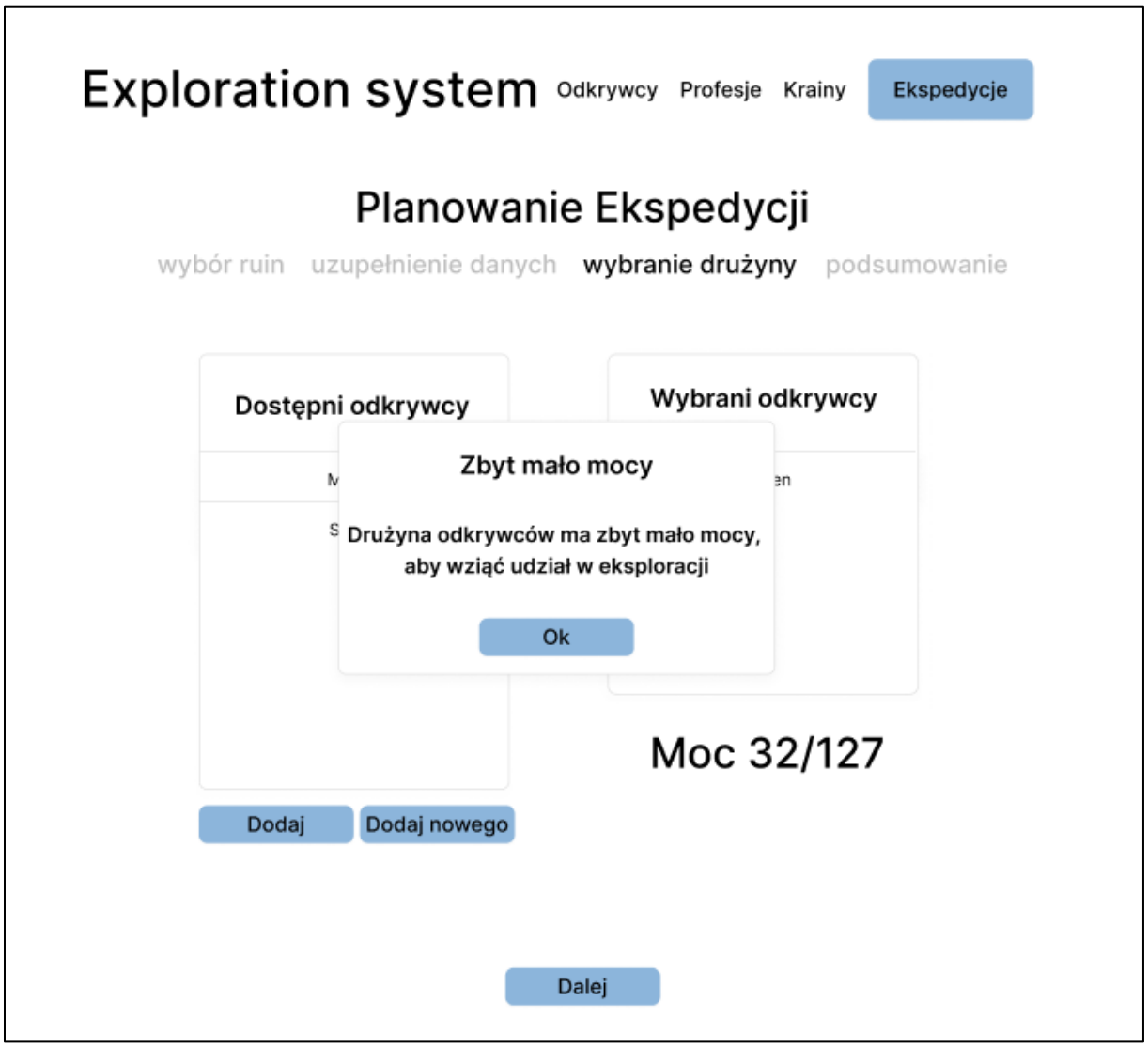
Rysunek 4: Widok planowania ekspedycji - uzupełnienie danych



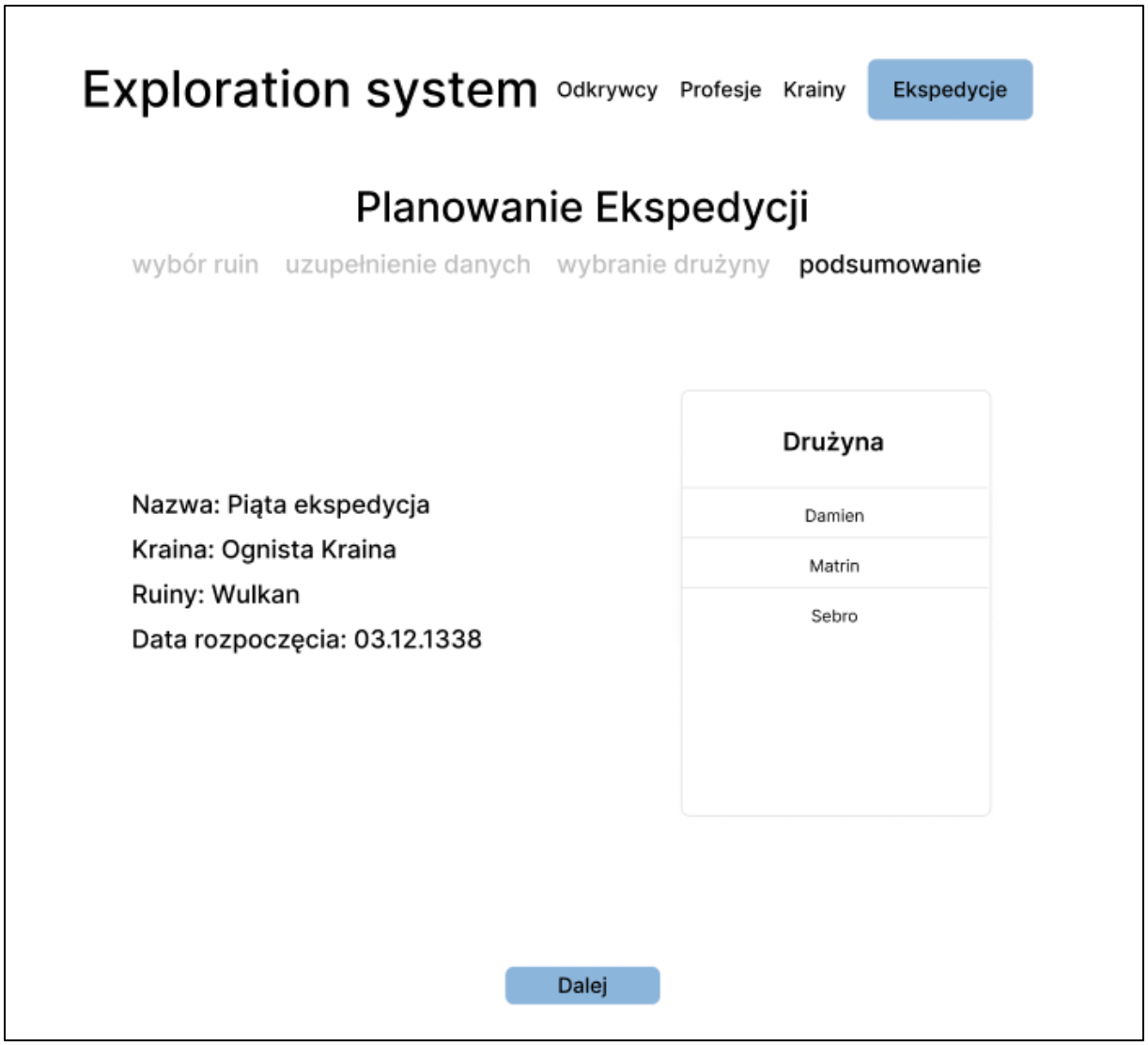
Rysunek 5: Widok planowania ekspedycji - wybór drużyny



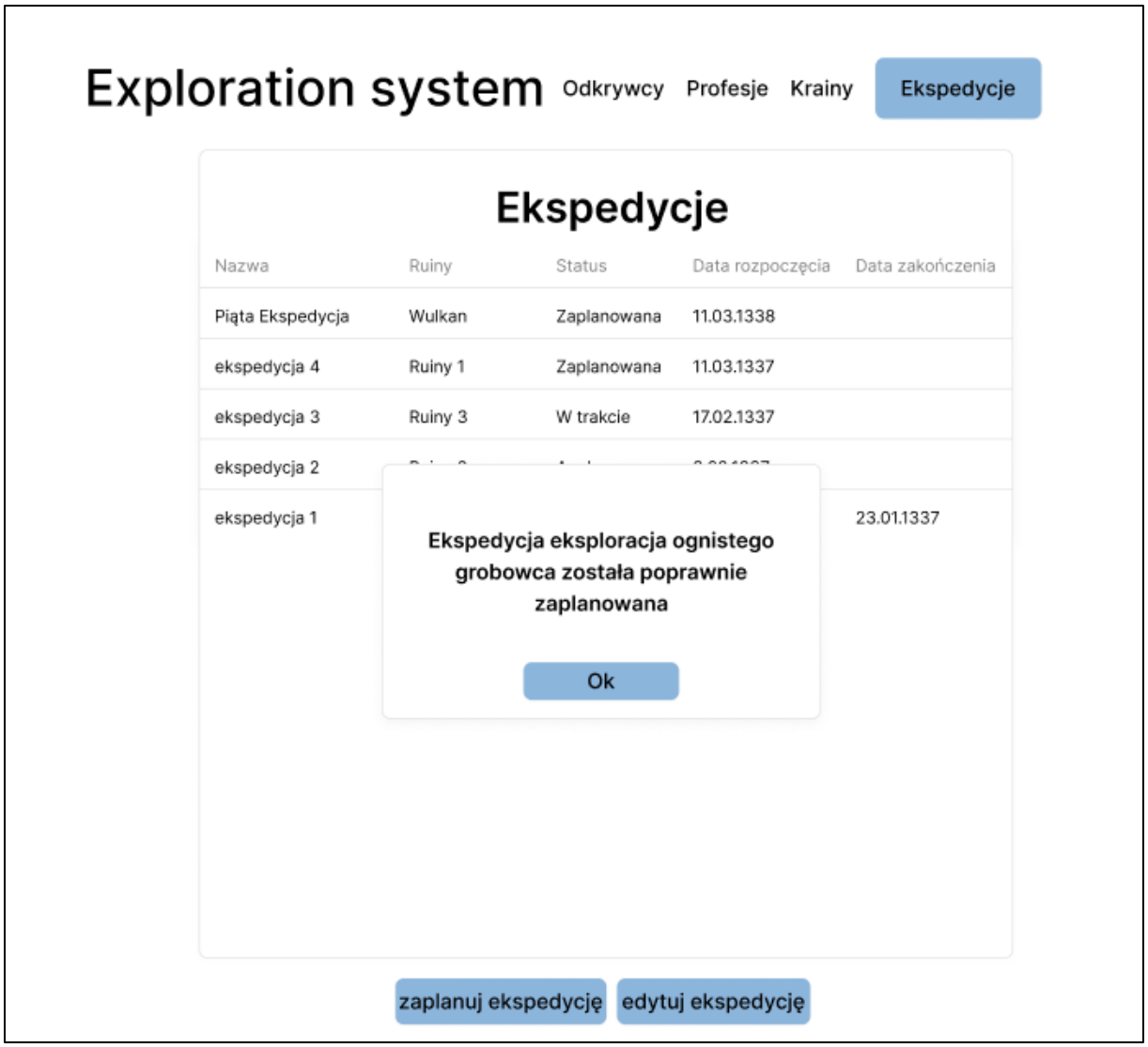
Rysunek 6: Widok planowania ekspedycji - wybór drużyny cz.2



Rysunek 7: Widok planowania ekspedycji - wybór drużyny - komunikat błędu



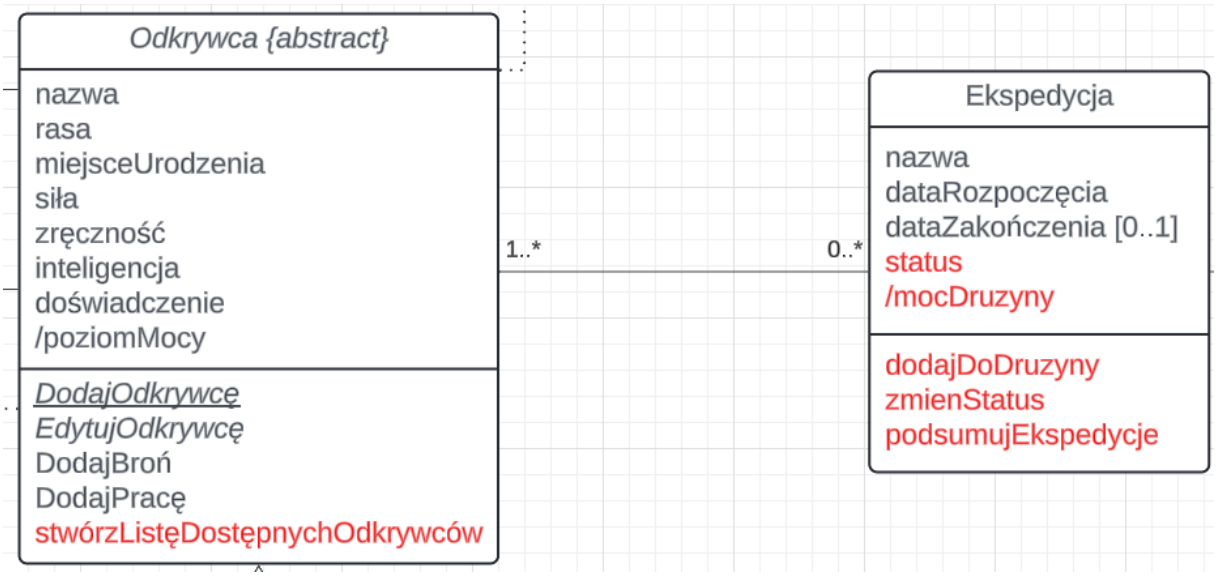
Rysunek 8: Widok planowania ekspedycji – podsumowanie



Rysunek 9: Widok listy ekspedycji – dodanie nowej ekspedycji

8. Analiza Dynamiczna

W wyniku analizy dynamicznej 2 klasy zostały zmienione.



Rysunek 10: Zmiany w wyniku analizy dynamicznej

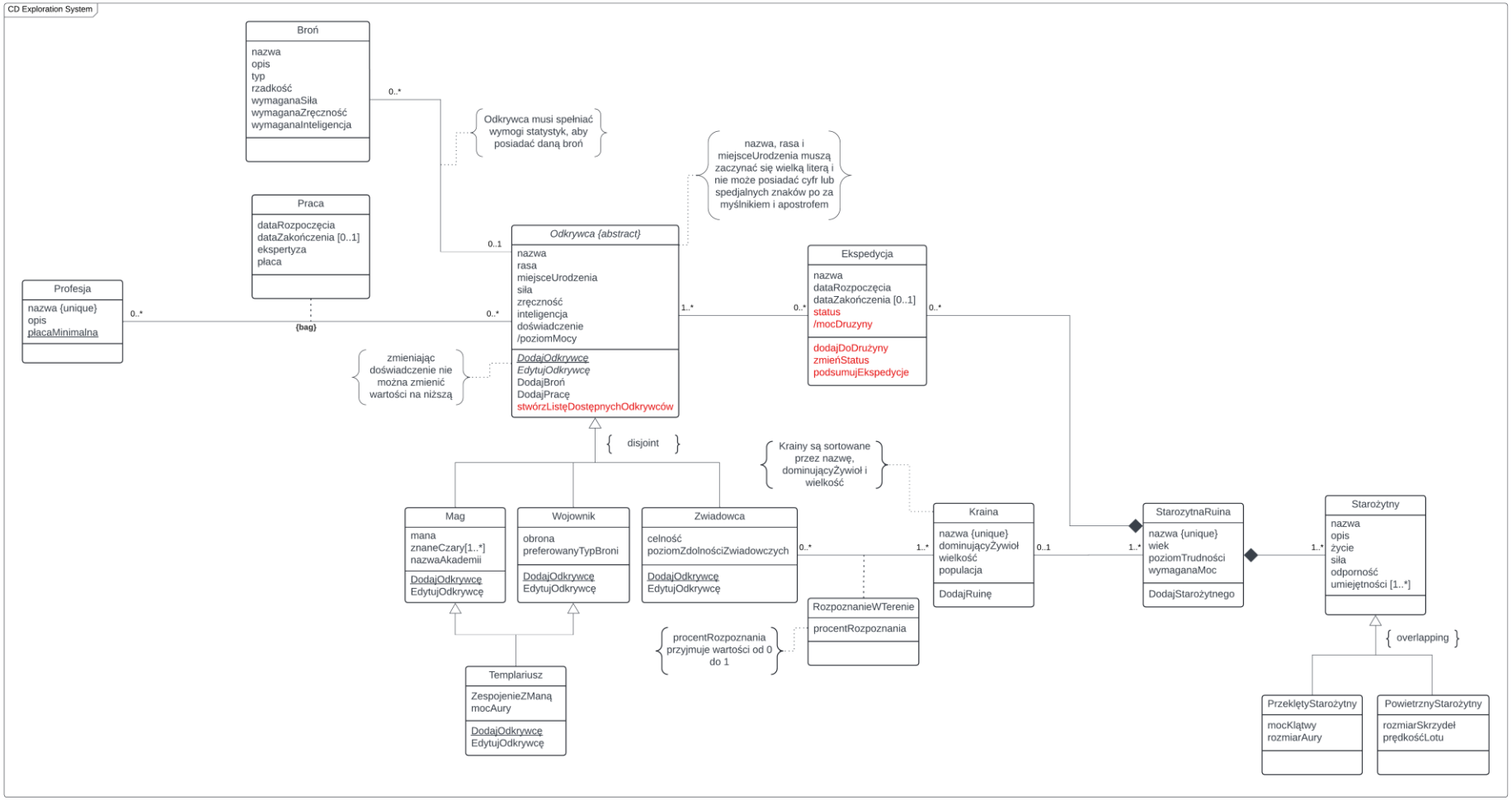
8.1. Aby móc śledzić status ekspedycji potrzebna jest nowa zmienna status oraz metoda zmieńStatus, aby móc ten status zmienić.

8.2. W celu zaplanowania ekspedycji potrzebujemy listę dostępnych odkrywców, czyli takich, którzy w obecnym momencie nie uczestniczą w żadnej ekspedycji (wszystkie ekspedycje, do których należą mają status zakończona/anulowana)

8.3. Potrzebujemy nowego atrybutu mocDrużyny, który będzie równy mocy wszystkich członków drużyny ekspedycyjnej, aby wiedzieć, czy drużyna spełnia wymogi eksplorowanej ruiny. Metoda dodajDoDrużyny pozwoli z kolei przy uzupełnianiu drużyny o nowych członków.

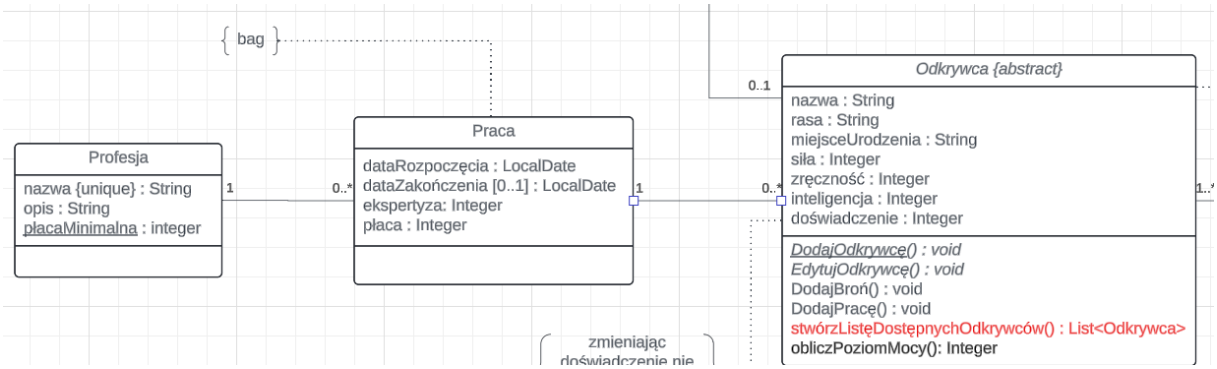
8.4. Metoda podsumujEkspedycje pozwoli na zdobycie wszystkich potrzebnych informacji do wyświetlenie podsumowania planowania ekspedycji

9. Analityczny diagram klas po analizie dynamicznej

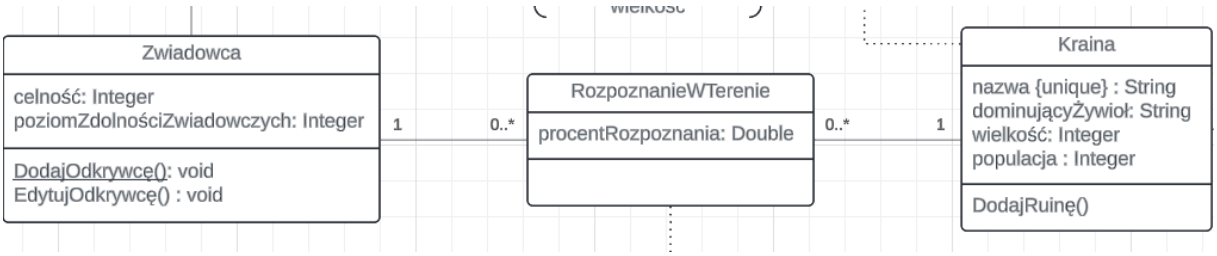


10.Decyzje projektowe

10.1. Asocjacja z atrybutem – aby zaimplementować asocjację z atrybutem trzeba zamienić klasę pośredniczącą w zwykłą klasę, która ma wymaganą asocjacje jeden do wielu do obu klas

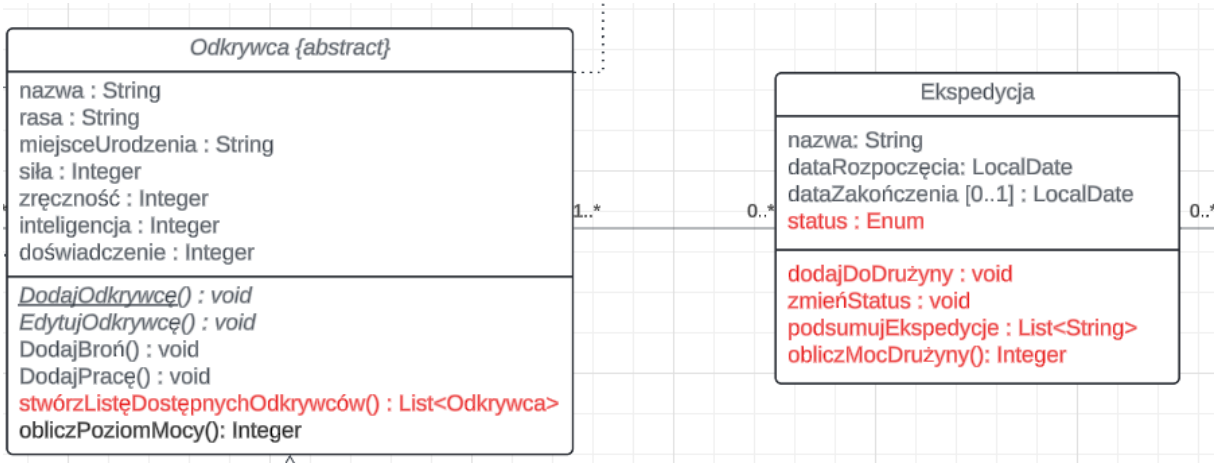


Rysunek 11: Implementacja asocjacji z atrybutem – profesja - odkrywca



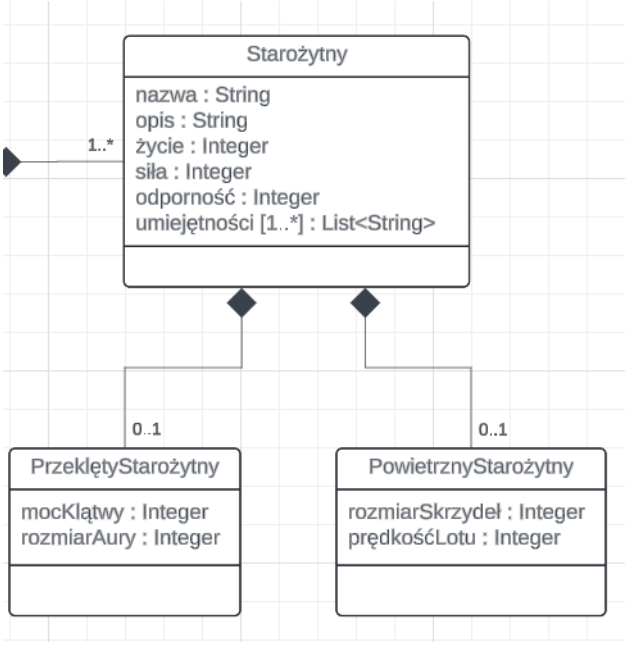
Rysunek 12: Implementacja asocjacji z atrybutem – zwiadowca – kraina

10.2. Atrybut pochodny – aby zaimplementować atrybut pochodny trzeba zamienić go na metodę



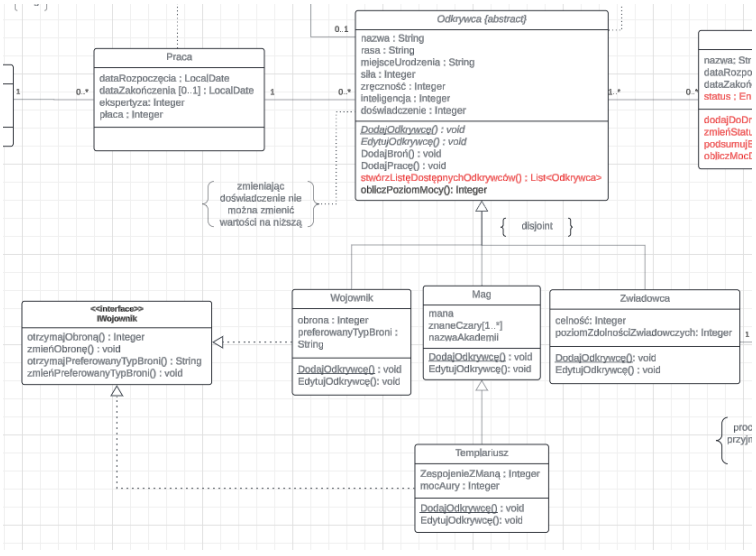
Rysunek 13: Zamiana atrybutu pochodnego na metodę

10.3. Dziedziczenie overlapping – aby zaimplementować dziedziczenie overlapping trzeba je zamienić na kompozycję jeden do jeden



Rysunek 14: Implementacja dziedziczenia overlapping

10.4. Wielodziedziczenie – aby zaimplementować wielodziedziczenie należy stworzyć interfejs, który będzie zawierał metody jednej z nadklas, po której podklasa dziedziczy. Wybrać należy mniej skomplikowaną klasę do tej operacji. Ten interfejs jest implementowany przez podklasę oraz nadklasę na bazie, której ten interfejs powstał



Rysunek 15: Implementacja wielodziedziczenia

10.5. Technologie wykorzystywane w projekcie

10.5.1. Java 20

10.5.2. Framework Spring

10.5.3. JavaFX

11. Projektowy diagram klas

